

p2 is a **sliced** Person **object**.

So this is just `Person::sleep()` .

Answer: 1

`p2.walk();`

Even though `walk()` is virtual, `p2` is no longer a `Student` ; slicing made it a plain `Person` .

Answer: 2

Final answer list for 21

- `p1->sleep();` → 1
 - `p1->walk();` → 5
 - `p1->eat();` → 3
 - `p1->study();` → WNC
 - `s1.sleep();` → 4
 - `s1.eat();` → 3
 - `p2.sleep();` → 1
 - `p2.walk();` → 2
-

Question 22 — how to approach it

This one has two parts:

1. Which pointer declarations compile
 2. What output each call gives
-

Step 1: know the class behavior

From the code:

```
class X
```

- `virtual a()` → prints `X::a`

- `b()` → prints `X::b` (**not virtual**)
- `virtual c()` → prints `X::c`
- `virtual c(int)` → prints `X::c(r)`
- `d()` → prints `X::d` (**not virtual**)

```
class Y : public X
```

- overrides `a()` → `Y::a`
- defines `b()` → `Y::b` (**but base `b` is not virtual**)
- defines `virtual d()` → `Y::d`
- defines `e()` → `Y::e`

```
class Z : public Y
```

- overrides `c()` → `Z::c`

Important traps:

- `a()` is virtual
- `b()` is not virtual
- `c()` is virtual
- `c(int)` is virtual, but **not overridden**
- `d()` in `X` is not virtual, so calling through `X*` still binds to `X::d`

Part A — which declarations compile

Rule

A base-class pointer can point to a derived object.

A derived-class pointer cannot point to a base object without a cast.

```
X * xx = new X;
```

Compiles

```
X * xy = new Y;
```

Compiles

```
X * xz = new Z;
```

Compiles

$Y * yx = \text{new } X;$

X is not necessarily a Y .

WNC

$Y * yy = \text{new } Y;$

Compiles

$Y * yz = \text{new } Z;$

A Z is a Y .

Compiles

$Z * zx = \text{new } X;$

Base to derived.

WNC

$Z * zz = \text{new } Z;$

Compiles

Final declaration answers

- $X * xx = \text{new } X; \rightarrow \text{OK}$
- $X * xy = \text{new } Y; \rightarrow \text{OK}$
- $X * xz = \text{new } Z; \rightarrow \text{OK}$
- $Y * yx = \text{new } X; \rightarrow \text{WNC}$
- $Y * yy = \text{new } Y; \rightarrow \text{OK}$
- $Y * yz = \text{new } Z; \rightarrow \text{OK}$
- $Z * zx = \text{new } X; \rightarrow \text{WNC}$
- $Z * zz = \text{new } Z; \rightarrow \text{OK}$

Part B — outputs

$xx \rightarrow a();$

`xx` is an `X*` to an `X` .

Virtual call, actual object is `X` .

Output: `X::a`

`xx->b()` ;

Not virtual, and object is `X` anyway.

Output: `X::b`

`xx->c()` ;

Virtual, actual object is `X` .

Output: `X::c`

`xy->a()` ;

`xy` is `X*` to `Y` .

`a()` is virtual, so actual object decides.

Output: `Y::a`

`xy->b()` ;

`b()` is **not virtual**.

So the static type `X*` decides.

Output: `X::b`

`xy->c()` ;

`c()` is virtual, but `Y` does **not** override `c()` .

So use `X::c` .

Output: `X::c`

`xy->c(10);`

Virtual, but neither `Y` nor `Z` override `c(int)` in this call context.

So it stays in `X`.

Output: `X::c(r)`

`xy->d();`

This is a trap.

`d()` in `X` is **not virtual**. Even though `Y` has its own `d()`, calling through an `X*` uses `X::d`.

Output: `X::d`

`xz->a();`

`xz` is `X*` to `Z`.

`Z` does not override `a()`, but inherits `Y::a`.

Output: `Y::a`

`xz->b();`

`b()` is not virtual, static type is `X*`.

Output: `X::b`

`xz->c();`

`c()` is virtual and `Z` overrides it.

Output: `Z::c`

```
yy->a();
```

yy is Y* to Y .

Output: Y::a

```
yy->c();
```

Y does not override c() .

Output: X::c

```
yz->a();
```

yz is Y* to Z .

a() is virtual. Z inherits Y::a .

Output: Y::a

```
yz->c();
```

c() is virtual and Z overrides it.

Output: Z::c

```
yz->c(20);
```

Careful: Z overrides only c() , not c(int) .

So for c(int) , it still uses X::c(r) .

Output: X::c(r)

```
zz->a();
```

zz is Z* to Z .

Z inherits Y::a .

Output: Y::a

zz->c();

Z overrides c() .

Output: Z::c

Final answer list for 22

Declarations

- X * xx = new X; → OK
- X * xy = new Y; → OK
- X * xz = new Z; → OK
- Y * yx = new X; → WNC
- Y * yy = new Y; → OK
- Y * yz = new Z; → OK
- Z * zx = new X; → WNC
- Z * zz = new Z; → OK

Outputs

- xx->a(); → X::a
- xx->b(); → X::b
- xx->c(); → X::c
- xy->a(); → Y::a
- xy->b(); → X::b
- xy->c(); → X::c
- xy->c(10); → X::c(r)
- xy->d(); → X::d
- xz->a(); → Y::a
- xz->b(); → X::b
- xz->c(); → Z::c
- yy->a(); → Y::a
- yy->c(); → X::c
- yz->a(); → Y::a

- `yz->c();` → `Z::c`
- `yz->c(20);` → `X::c(r)`
- `zz->a();` → `Y::a`
- `zz->c();` → `Z::c`

Fast exam method for these questions

For every line, ask in this order:

For declarations

1. Is pointer type base or derived?
2. Base pointer to derived object = okay
3. Derived pointer to base object = not okay

For method calls

1. Does the static type have the method?
2. Is it virtual?
3. If virtual → use actual object
4. If not virtual → use pointer/reference type
5. Watch for slicing

The traps they want you to miss

- non-virtual methods called through base pointers
- overloaded virtuals like `c()` vs `c(int)`
- object slicing
- calling derived-only methods through base pointers

The cleanest next step is to drill these by making a one-page “dispatch rules” sheet and memorizing it.

Sources

